Assignment 2 — Household Object Recognition, Levelled Up: Text-to-Video Synthesis for Robustness, CLIP Quality Gate and an Explainable Demo App

Nurlan Bekov
Raiymbek Mussin
Computer Vision · Astana IT University · November 2026

Summary

We extend the A1 classifier for cups, bottles, keys and pens with a state-of-the-art text-to-video generator. CogVideoX-2B, an open alternative to Step-Video-T2V that fits a free Colab T4, produced 16 clips with camera motion, blur and new backgrounds. A CLIP quality gate filtered the frames sampled from them, and the kept frames were added to training. Everything was evaluated with 5-fold cross-validation on the 115 real photos, on clean images and under 7 phone-style corruptions.

The best trained model, **EfficientNet-B0 with synthetic frames, reached 97.4% clean accuracy and 86.2% stress-test accuracy**. The A1 baseline (ResNet-18) reached 93.9% and 78.9%. Most of that gain came from the backbone; synthetic frames added +0.9 pp clean and +1.0 pp stress over the same backbone without them. For ResNet-18, CLIP-gated synthetic frames did not help (91.3% clean): poorly generated "keys" taught the model to confuse keys with pens. A zero-shot CLIP ViT-B/32 with no training was the strongest model overall (100% clean, 95.0% stress). The demo app shows predictions, Grad-CAM, nearest neighbours, latency and the environment, and labels every synthetic clip.

1. Dataset update

The real data are the A1 iPhone photos of the authors' own objects. They were cleaned (EXIF orientation, corrupted-file check, dHash near-duplicate removal) and **balanced to at most 30 photos per class**.

Class	Real photos	Synthetic frames sampled	Synthetic frames kept by CLIP	Synthetic share of training pool
bottle	27	10	8	22.9%
cup	30	14	10	25.0%
key	30	22	8	21.1%
pen	28	15	15	34.9%
Total	**115**	**61**	**41**	**26.3%**

Each of the 16 clips (4 per class) has 49 frames; 8 frames were sampled per clip, and exact repeats were dropped. CogVideoX produced clips with little motion, so only 61 of 128 sampled frames were distinct. Training augmentation is unchanged from A1 and applied on the fly to training images only: random resized crop, horizontal flip, ±20° rotation, colour jitter, random grayscale and random erasing. **Synthetic frames are used for training only.** Every score in this report is measured on real photos.

2. SOTA method: text-to-video synthesis for robustness

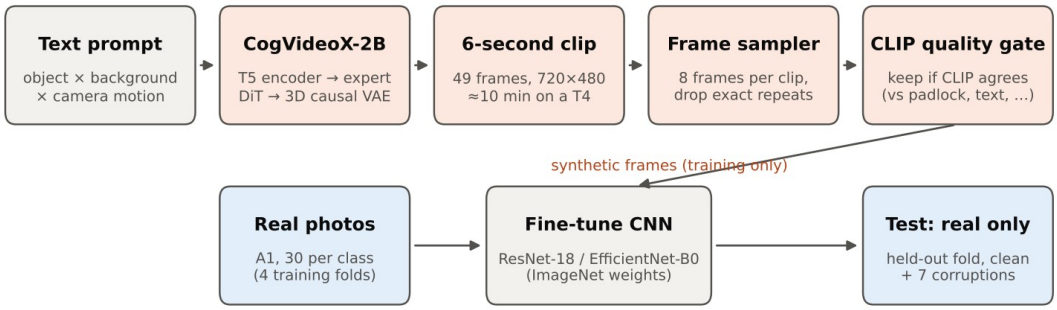


Figure 1. Pipeline: prompt → video → frames → CLIP gate → training; evaluation on real photos only.

What the assignment proposes. For our topic, the brief recommends **Step-Video-T2V**, a 30-billion-parameter open-source text-to-video model (Neurohive; arXiv 2502.10248). Its Video-VAE compresses video 16×16 in space and 8× in time. A diffusion transformer (48 layers, 3D full attention) then denoises in that compact latent space,

conditioned on two bilingual text encoders. It generates up to 204 frames. The official repository reports 72–79 GB peak GPU memory per video and recommends 80 GB GPUs (GitHub). That cannot run on the free Colab T4 (15 GB) available to us.

What we used. CogVideoX-2B (arXiv 2408.06072, ICLR 2025) follows the same recipe at a smaller scale. A T5 text encoder feeds an *expert transformer*: text and video tokens share one sequence, and expert adaptive LayerNorm handles each modality. A 3D causal VAE compresses video 4× in time and 8×8 in space, and 3D full attention keeps motion consistent. With 2 billion parameters, fp16 weights and CPU offloading of idle parts, it fits on a T4. Each 6-second, 49-frame, 720×480 clip took **10.0 minutes** at 30 denoising steps, or 2.7 GPU-hours for all 16 clips. We credit Step-Video-T2V for the idea and used CogVideoX-2B as the open model we could actually run. The second suggested method, ArtAug (Neurohive), builds on the 12-billion-parameter FLUX model and was out of reach for the same reason.

Why it should help this topic. A1 revealed a shortcut. Bottles and cups were photographed standing in the kitchen, keys and pens lying flat on fabric or floors, so a model could partly recognise the *scene*. Our prompts combine **object variant × background × camera motion**, for example: *"A handheld phone camera slowly orbits around a single brass house key, macro close-up, on a grey carpet. Realistic home video, slight motion blur…"*. Every class therefore appears on kitchen tables, carpets, bed sheets, desks and in a hand. The prompts also ask for camera motion and blur that the still photos lack.

CLIP quality gate. Generated frames are not always correct (Figure 2). In the first run, CogVideoX drew padlocks and a metal block with a fake stock-footage logo when asked for keys. CLIP ViT-B/32 now scores every frame against the four class prompts *and* six "none of these" prompts (padlock, lock, metal box, text, blurry dark photo, empty table). A frame is kept only if its intended class wins with probability ≥ 0.6. The gate kept 41 of 61 frames, but only 8 of 22 for keys (36%).



Figure 2. Middle frames of four CogVideoX-2B clips (all synthetic).

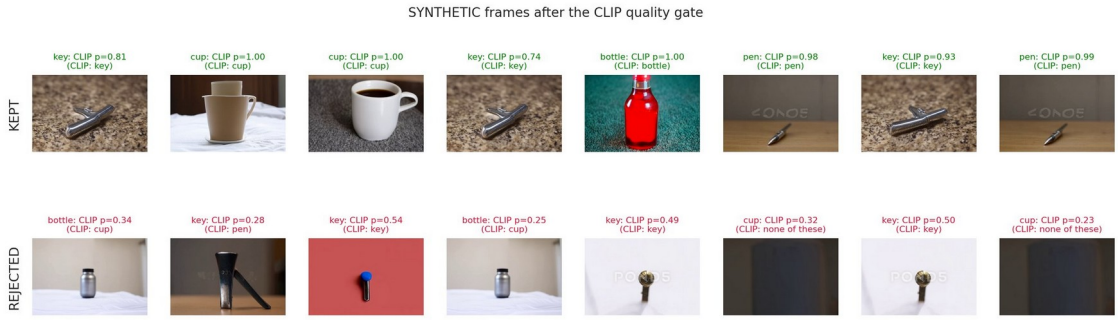


Figure 3. CLIP quality gate: kept (top) and rejected (bottom) synthetic frames with CLIP's probability for the intended class.

3. Experiments

Protocol. We used stratified 5-fold cross-validation on the 115 real photos, so every photo is tested exactly once (about 23 per fold). Within each training part, 15% of the real photos form a validation set for early stopping. Synthetic frames are added only to the training part. Backbones are ImageNet-pretrained ResNet-18 and EfficientNet-B0, trained in two stages: a frozen-backbone head for 5 epochs, then full fine-tuning for up to 10 epochs. We used AdamW, label smoothing 0.1, batch size 32 and seed 42. **Stress test:** every held-out photo is also evaluated under 7 deterministic corruptions: motion blur, defocus blur, low light with noise, 60° rotation, perspective (viewpoint) warp, 45% occlusion and JPEG quality 8. We report accuracy, macro-F1 and mAP (macro average precision over the four classes, computed from class probabilities). This is a classification task, so there are no boxes and no detection mAP@0.5.

Model	Clean Acc	Clean F1	F1 fold SD	Clean mAP	Stress Acc	Stress F1
A1 classical: HOG + SVM	0.791	0.783	—	0.887	0.617	0.621
A1 baseline: ResNet-18, real only	0.939	0.939	0.045	0.987	0.789	0.788
ResNet-18 + synthetic (CLIP-gated)	0.913	0.913	0.070	0.991	0.770	0.764
EfficientNet-B0, real only (control)	0.965	0.965	0.044	0.997	0.852	0.853
EfficientNet-B0 + synthetic (CLIP-gated)	**0.974**	**0.973**	0.037	0.990	**0.862**	**0.863**
Zero-shot CLIP ViT-B/32 (no training)	1.000	1.000	—	1.000	0.950	0.950

The first five rows are trained models; the last is a zero-shot reference. Stress values are averaged over the 7 corruptions (805 predictions). Figure 4 breaks the stress test down by corruption for the A1 baseline and the two EfficientNet models.

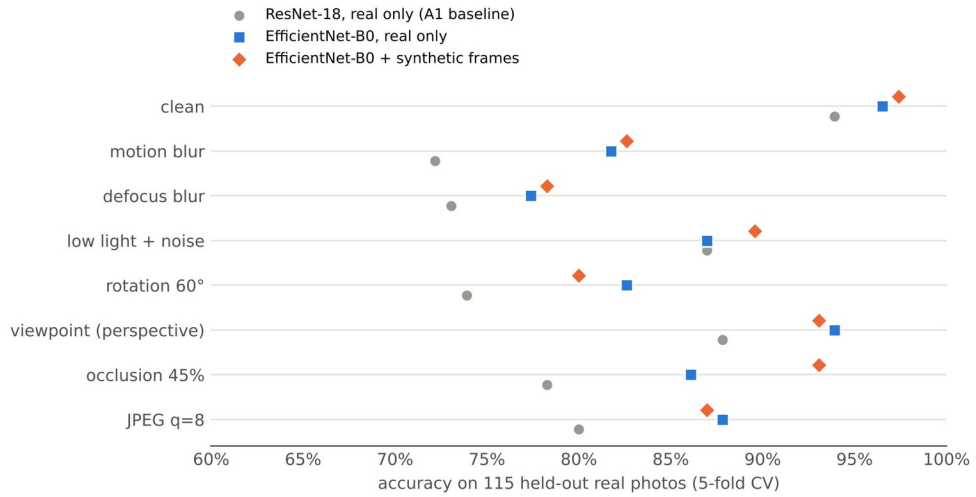


Figure 4. Accuracy per corruption (5-fold CV on 115 real photos).

Reading the results. Moving from ResNet-18 to EfficientNet-B0 gave the largest gain: +2.6 pp clean and +6.3 pp stress. Synthetic frames added a smaller amount on top of EfficientNet: +0.9 pp clean and +1.0 pp stress overall. The largest per-corruption gains were occlusion (+7.0 pp, 86.1% → 93.0%) and low light (+2.6 pp); rotation dropped by 2.6 pp. These differences are 1–3 images out of 115, while the fold-to-fold standard deviation of F1 is 4–7 pp, so we treat them as indicative, not significant. Zero-shot CLIP, trained on 400 million image–text pairs, beat every model we fine-tuned on 115 photos, especially on blur and low light.

Ablation (one change at a time, ResNet-18):

Variant	Clean Acc	Stress Acc	Δ stress vs A1 baseline
A1 baseline (real only, augmentation on)	0.939	0.789	—
+ synthetic frames, CLIP-gated (41 frames)	0.913	0.770	−1.9 pp
+ synthetic frames, no gate (61 frames)	0.957	0.810	+2.1 pp
real only, augmentation off	0.948	0.824	+3.5 pp

The gate made ResNet-18 *worse*, not better. Figure 6 shows why: with gated synthetic frames, ResNet-18 labelled 6 of 30 clean keys as pens (2 for the baseline) and 15 of 30 under motion blur. The gate threw out padlocks but kept elongated metal "keys" without a clear bow or teeth (Figure 3, top), which look like the metal tips in the pen clips. Wrong-looking frames for the hardest class hurt more than the gate helped. Without the

gate the extra frames did help, but the difference (+2.1 pp) is again within noise. Turning augmentation off did not hurt either. On 92 training photos with a 14-image validation set, validation accuracy reaches 100% within 4–5 epochs (Figure 5), so early stopping has almost no signal and the variance between runs is large.

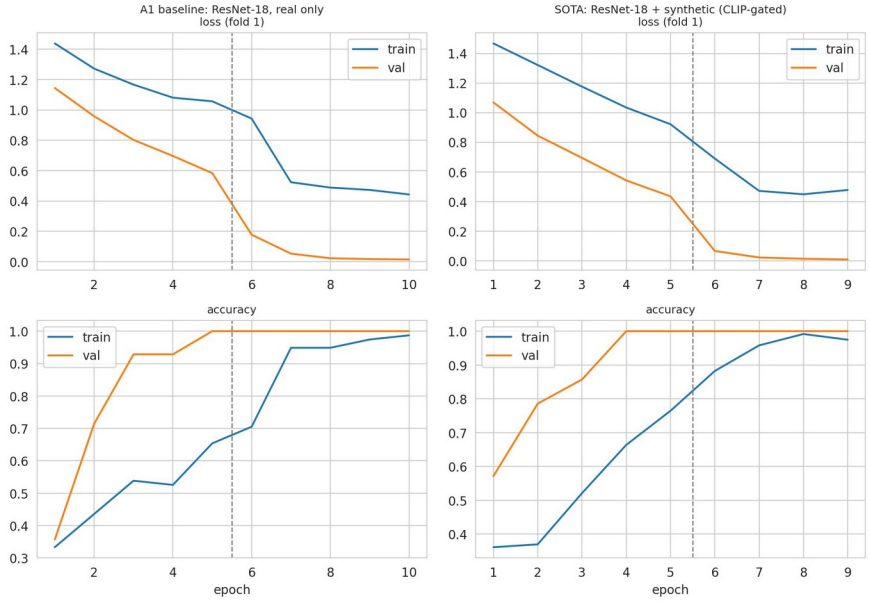


Figure 5. Training curves, fold 1: A1 baseline (left) vs + synthetic frames (right). The dashed line marks the start of full fine-tuning.

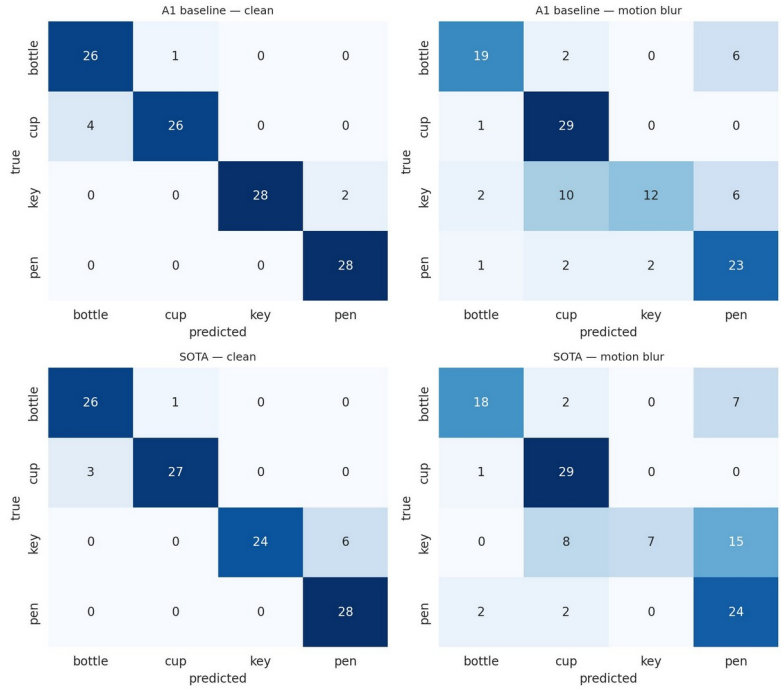


Figure 6. Confusion matrices over all 115 photos for ResNet-18: A1 baseline (top) and + gated synthetic frames (bottom), clean (left) and motion blur (right).

Speed (batch 1, 224 × 224):

Backbone	Tesla T4	FPS (T4)	Colab CPU	FPS (CPU)
ResNet-18	4.7 ms	213	60.2 ms	17
EfficientNet-B0	8.6 ms	117	36.0 ms	28

Both models run in real time on a GPU. On a CPU, EfficientNet-B0 is the faster and the more accurate choice. Generating training data is the expensive step: about 10 minutes of T4 time per 6-second clip.

4. Explainability

The app and the notebook use **Grad-CAM** on the last convolutional block, plus a **nearest-neighbour gallery** in the model's 512-dimensional embedding space. On clean photos, the heat-map sits on the object: the bottle

body, the cup handle, the key bow and the pen barrel (Figure 7, top row). Under motion blur the bottle, cup and pen are still found. The blurred key is predicted as *pen*, and its heat-map spreads over the background: the model loses the key's shape and falls back on context. This failure matches the key→pen confusions in Figure 6. In the nearest-neighbour example (Figure 8), all six nearest training images of a held-out glass mug are mugs. The first is the query photo itself (similarity 1.00), because the app's gallery holds every training image, and the next are other photos of the same and similar glass mugs.

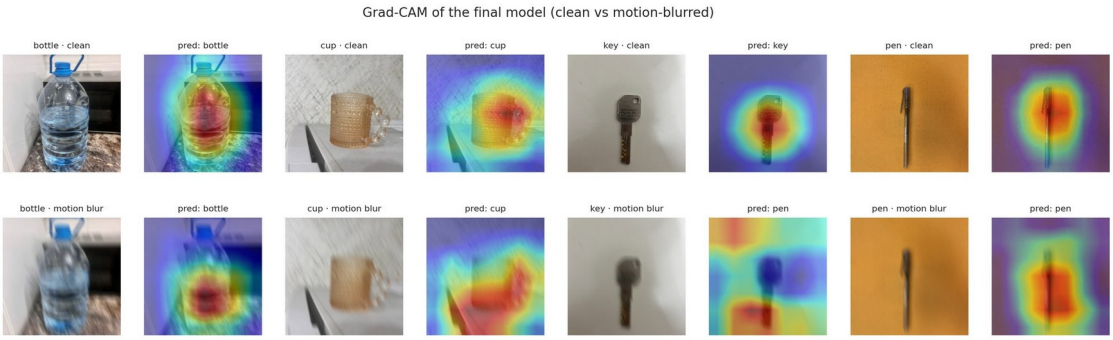


Figure 7. Grad-CAM of the final ResNet-18 + synthetic model: clean (top) and motion-blurred (bottom) test photos.



Figure 8. Nearest neighbours of a query photo in the training gallery (real photos and labelled synthetic frames).

Demo app. The app is written in Gradio (`app.py`) and has four tabs. *Recognize*: upload or camera → class probabilities, Grad-CAM, 6 nearest neighbours tagged *real* or *SYNTHETIC*, latency of the request, session average and FPS, and the server device and the user's browser. *Stress test*: apply any of the 7 corruptions and re-predict. *Synthetic data*: the generated clips, watermarked "SYNTHETIC · generated with CogVideoX-2B", each shown with its prompt. *About & ethics*.

5. Ethics and limitations

- **Disclosure and watermarking.** All generated media are labelled and carry a red *SYNTHETIC* watermark in the app. They are used only for training and never for evaluation.
- **Consent.** The real photos show only the authors' own objects at home: no people, faces, documents or addresses. Uploaded images are processed in memory and not stored.
- **Provenance of generated data.** Several CogVideoX frames contain a fake stock-footage watermark ("PONDS", "<ONOE", Figure 3). This suggests that the generator learned from watermarked stock video. Such frames may carry copyright or licensing concerns, and they inject text artifacts into training data. The CLIP gate's "text" prompt removes some but not all of them.
- **Bias and validity.** One home, one phone and 27–30 photos per class: accuracy on other homes and cameras will be lower. Object instances were not recorded, so photos of the same object can fall into both training and test folds. The A1 scene shortcut (kitchen vs floor) is reduced by the prompts but still present in the real test photos.
- **Misuse.** The risk is low for household objects. The same synthetic-data pipeline could, however, fabricate misleading video, which is why we disclose and watermark it.
- **Limitations.** The synthetic set is small (41 frames after the gate). The generator is weak on small, thin objects (keys), and the clips show little motion. A 30-billion-parameter model such as Step-Video-T2V, image-to-video conditioning on our own photos, or a larger CLIP for gating would likely give better frames.

6. Reproducibility

- **Environment.** Google Colab, Tesla T4 (15 GB), Python 3.13, PyTorch 2.11 (CUDA 13.0), diffusers ≥ 0.30, transformers, scikit-learn. Seed 42 everywhere. The 5-fold split, every prompt and every per-clip seed are saved.
- **Weights (downloaded automatically).** `THUDM/CogVideoX-2b` (Apache-2.0), `openai/clip-vit-base-patch32`, torchvision ImageNet weights for ResNet-18 and EfficientNet-B0.
- **Inputs.** Photos in `MyDrive/household_objects/raw/{bottle,cup,key,pen}/`. Dataset: Google Drive.
- **Run the experiments.** Open `notebooks/A2_household_objects_SOTA.ipynb` in Colab, select *T4 GPU* and choose *Run all*. Generation takes about 2.7 h. Clips are saved one by one, so a restarted run skips finished clips. Outputs: `a2/results/` (tables, figures, `a2_metrics.json`) and `a2/app_bundle.zip`.
- **Run the app.** `pip install -r requirements.txt`, then `python app.py --bundle app_bundle` (http://127.0.0.1:7860). In Colab, add `--share` for a public link.

References

1. StepFun. Step-Video-T2V Technical Report. arXiv:2502.10248, 2025. https://arxiv.org/abs/2502.10248
2. Neurohive. Step-Video-T2V: Text-to-Video Open-Source Model Achieves 16x Video Compression Breakthrough. https://neurohive.io/en/state-of-the-art/step-video-t2v-text-to-video-open-source-model-achieves-16x-video-compression-breakthrough/
3. Z. Yang et al. CogVideoX: Text-to-Video Diffusion Models with an Expert Transformer. ICLR 2025. https://arxiv.org/abs/2408.06072
4. Neurohive. ArtAug: Open Source Framework for Image Generation Models Enhancement. https://neurohive.io/en/state-of-the-art/artaug-a-novel-synthesis-understanding-framework-for-image-generation-enhancement/
5. A. Radford et al. Learning transferable visual models from natural language supervision (CLIP). ICML 2021. https://arxiv.org/abs/2103.00020
6. K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. CVPR 2016. https://doi.org/10.1109/CVPR.2016.90
7. M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. ICML 2019. https://arxiv.org/abs/1905.11946
8. R. R. Selvaraju et al. Grad-CAM: Visual explanations from deep networks via gradient-based localization. ICCV 2017. https://doi.org/10.1109/ICCV.2017.74
9. D. Hendrycks and T. Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. ICLR 2019. https://arxiv.org/abs/1903.12261